

Boot Spring Boot

부트 스프링 부트

Jake Jiheon Kim

Version 0.0.1, 2016-09-01

Table of Contents

저자 소개	2
들어가며.....	3
책에서 다루는 내용	3
책을 따라하기 위해 필요항목.....	3
대상독자.....	3
관례(Convention)	3
예제	5
스프링 부트	6
1. 시작하기.....	7
1.1. 스프링 부트 소개.....	7
1.2. 시스템 요구사항	7
1.3. 스프링 부트 설치.....	7
1.3.1. 설치방법 소개	7
1.4. 스프링 부트 애플리케이션 개발	7
1.4.1. 프로젝트 생성	7
1.4.2. 의존성 추가	7
1.4.3. 코드 작성하기	7
1.4.4. 실행하기.....	7
1.4.5. 실행가능한 jar 생성.....	7
1.5. 이어질 이야기	7
2. 스프링 부트 이용	8
2.1. 빌드 시스템.....	8
2.1.1. 의존성 관리	8
2.1.2. 메이븐	8
2.1.3. 그레이들.....	8
2.1.4. 스타터	8
2.2. 코드 구조.....	8
2.2.1. "기본" 패키지 사용	8
2.2.2. 메인 애플리케이션 클래스 위치.....	8
2.3. 구성(Configuration) 클래스	8
2.3.1. 추가적인 구성클래스 불러오기	8
2.3.2. XML 구성 불러오기	8
2.4. 자동구성(Auto-configuration)	8
2.4.1. 자동구성 대체하기	8
2.4.2. 특정 자동구성 비활성화하기	8
2.5. 스프링 빈과 의존성 주입.....	8
2.6. @SpringBootApplication 애노테이션 사용하기.....	8
2.7. 애플리케이션 실행하기.....	8
2.7.1. IDE 에서 실행하기	8
2.7.2. 패키징된 애플리케이션 실행하기	8
2.7.3. 메이븐 플러그인으로 실행하기	9
2.7.4. 그레이들 플러그인으로 실행하기	9
2.7.5. 핫 스와핑	9
2.8. 스프링 개발자도구	9
2.8.1. 기본 속성들	9

2.8.2. 자동 재시작	9
2.8.3. 전역 설정	9
2.8.4. 원격 애플리케이션	9
2.9. 출시를 위한 애플리케이션 패키징	9
2.10. 이어질 이야기	9
3. 스프링 부트 기능	10
3.1. 스프링애플리케이션(SpringApplication)	10
3.1.1. 배너 재정의	10
3.1.2. SpringApplication 재정의	10
3.1.3. 유연한(Fluent) 빌더 API	10
3.1.4. 애플리케이션 이벤트 및 리스너	10
3.1.5. 웹 환경	10
3.1.6. 애플리케이션 실행 인자에 접근	10
3.1.7. ApplicationRunner 혹은 CommandLineRunner 사용하기	10
3.1.8. 애플리케이션 종료	10
3.1.9. 관리자 기능	10
3.2. 외부 구성	10
3.2.1. 랜덤값 구성	10
3.2.2. 명령줄 속성 접근	10
3.2.3. 애플리케이션 속성 파일	10
3.2.4. 프로파일 정의 속성	10
3.2.5. 속성 내 치환자(placeholder)	10
3.2.6. properties를 대신하여 YAML 사용하기	10
3.2.7. 보장형 구성 속성	11
3.3. 프로파일	11
3.3.1. 활성화할 프로파일 추가하기	11
3.3.2. 프로파일을 프로그래밍적으로 설정	11
3.3.3. 프로파일 정의 구성 파일	11
3.4. 로깅	11
3.4.1. 로그 형식	11
3.4.2. 콘솔 출력	11
3.4.3. 파일 출력	11
3.4.4. 로그 레벨	11
3.4.5. 로그 구성 재정의	11
3.4.6. 로그백 확장	11
3.5. 웹애플리케이션 개발	11
3.5.1. 'Spring Web MVC framework'	11
3.5.2. 내장 서블릿 컨테이너 지원	11
3.6. 보안	12
3.6.1. OAuth2	12
3.6.2. 사용자 정보 속 토큰 유형	12
3.6.3. 사용자정보 RestTemplate 재정의	12
3.6.4. 액추에이터 보안(Actuator Security)	12
3.7. SQL 데이터베이스 이용	12
3.7.1. DataSource 구성	12
3.7.2. JdbcTemplate 사용	12
3.7.3. JPA 그리고 '스프링 Data'	12

3.7.4. H2 사용	12
3.7.5. jOOQ 사용	12
3.8. NoSQL 기술 이용	13
3.8.1. 레디스(Redis)	13
3.8.2. 몽고DB(MongoDB)	13
3.8.3. 솔(Solr)	13
3.8.4. 엘라스틱서치(Elasticsearch)	13
3.9. 캐시	13
3.9.1. 지원하는 캐시 제공자(Provider)	13
3.10. REST 서비스 호출	14
3.10.1. RestTemplate 재정의	14
3.11. 스프링 세션	14
3.12. 테스트	14
3.12.1. 테스트 관련 의존성	14
3.12.2. 스프링 애플리케이션 테스트	14
3.12.3. 스프링 부트 애플리케이션 테스트	14
3.12.4. 테스트 유틸리티	14
3.13. 웹소켓(WebSocket)	15
3.14. 웹 서비스(Web Service)	15
3.15. 자동구성 직접 생성하기	15
3.15.1. 자동구성 빈 이해하기	15
3.15.2. 자동구성 대상자 위치	15
3.15.3. 조건 애너테이션	15
3.15.4. 스타터 직접 생성하기	15
3.16. 이어질 이야기	15
4. 스프링 부트 액츄에이터: 출시준비기능	16
4.1. 출시준비 기능 활성화	16
4.1.1. 종단점(Endpoint)	16
4.1.2. 종단점 재정의	16
4.1.3. 액츄에이터 MVC 종단점 를 위한 하이퍼미디어	16
4.1.4. CORS 지원	16
4.1.5. 재정의 종단점 추가하기	16
4.1.6. 건강정보	16
4.1.7. HealthIndicator 를 사용한 보안	16
4.1.8. 애플리케이션 정보	16
4.2. HTTP 로 모니터링하고 관리하기	16
4.2.1. 종단점 에 대한 노출강도 보안관리	16
4.2.2. 관리 종단점 경로 재정의	16
4.2.3. 관리 서버 포트 재정의	16
4.2.4. 관리를 위한 SSL 재정의	16
4.2.5. 관리 서버 주소 재정의	16
4.2.6. HTTP 종단점 비활성화	16
4.2.7. HTTP 건강 종단점 접근 제한	17
4.3. 원격셀을 통해 모니터링하고 관리하기	17
4.3.1. 원격셀로 접속	17
4.3.2. 원격셀 확장	17
4.3.3. 원격셀 플러그인	17
4.4. 측량(Metrics)	17

4.4.1. 시스템 측량	17
4.4.2. 데이터소스 측량	17
4.4.3. 캐시 측량	17
4.4.4. 톨캣 세션 측량	17
4.4.5. 측량 기록	17
4.4.6. 공개 측량 추가	17
4.5. 감시	17
4.6. 추적	17
4.7. 프로세스 모니터링	17
4.7.1. 확장 구성	17
4.7.2. 프로그래밍적인 처리	17
4.8. 이어질 이야기	17
5. 스프링 부트 애플리케이션 배포	18
5.1. 클라우드 배포	18
5.1.1. 클라우드 파운드리(Cloud Foundry)	18
5.1.2. 히로쿠(Heroku)	18
5.1.3. 오픈시프트(OpenShift)	18
5.1.4. 아마존웹서비스(AWS, Amazon Web Service)	18
5.1.5. 구글앱엔진	18
5.2. 스프링 부트 애플리케이션 설치	18
5.2.1. 유닉스/리눅스 서비스	18
5.3. 마이크로소프트 윈도우즈 서비스	18
5.4. 이어질 이야기	18
6. 빌드 도구 플러그인	19
6.1. 스프링 부트 메이븐 플러그인	19
6.1.1. 플러그인 포함	19
6.1.2. 실행가능한 jar 와 war 파일 패키징	19
6.2. 스프링 부트 그레이들 플러그인	19
6.2.1. 플러그인 포함	19
6.2.2. 그레이들 의존성 관리	19
6.2.3. 실행가능한 jar 와 war 파일 생성	19
6.2.4. 스프링 부트 플러그인 구성	19
6.2.5. 리패키징 구성	19
6.2.6. 그레이들 플러그인이 어떻게 동작하는지 이해하기	19
6.2.7. 그레이들 을 이용해서 메이븐 리파지토리에 아티팩트 배포	19
6.3. 다른 빌드 시스템 지원	19
6.3.1. 리패키징 아카이브	19
6.3.2. 내포된 라이브러리	19
6.3.3. 메인 클래스 찾기	19
6.3.4. 리패키지 구현 예제	19
6.4. 이어질 이야기	19
7. 스프링 부트 활용 안내	20
7.1. 스프링 부트 애플리케이션	20
7.1.1. 자동구성 문제해결	20
7.1.2. Environment 혹은 ApplicationContext 가 시작하기 전에 재정의하기	20
7.1.3. ApplicationContext 계층 조직(부모 혹은 루트 컨텍스트 추가)	20
7.1.4. 비-웹애플리케이션 생성	20
7.2. 속성 및 구성	20

7.2.1. 빌드 시 자동으로 속성 확장	20
7.2.2. SpringApplication 외부 구성	20
7.2.3. 애플리케이션 속성 외장파일 위치변경	20
7.2.4. 명령줄 인자 축약사용	20
7.2.5. 외부속성에 대한 YAML 사용	20
7.2.6. 스프링 프로파일 활성화	20
7.2.7. 환경 의존적인 구성 변경	20
7.2.8. 외부속성에 대한 빌트인 옵션(built-in option) 발견	20
7.3. 내장서블릿 컨테이너	20
7.3.1. 애플리케이션에 서블릿, 필터 혹은 리스너 추가하기	20
7.3.2. HTTP 포트 변경	20
7.3.3. 할당되지 않은 HTTP 포트 무작위 사용	20
7.3.4. 실행시 HTTP 포트 발견	20
7.3.5. SSL 구성	20
7.3.6. 접근로그 구성	21
7.3.7. 프론트엔드 프록시 서버 이용	21
7.3.8. 톰캣 구성	21
7.3.9. 톰캣을 통한 다중 커넥터 활성화	21
7.3.10. 톰캣 7.x 혹은 8.0 이용	21
7.3.11. @ServerEndpoint 를 사용한 웹소켓 종단점 생성	21
7.3.12. HTTP 응답 압축 활성화	21
7.4. 스프링 MVC	21
7.4.1. JSON REST 서비스 작성	21
7.4.2. XML REST 서비스 작성	21
7.4.3. 잭슨(Jackson) 오브젝트매퍼(ObjectMapper) 재정의	21
7.4.4. @ResponseBody 렌더링 재정의	21
7.4.5. Multipart 파일 업로드 제어	21
7.4.6. 스프링 MVC DispatcherServlet 전환	21
7.4.7. 기본 MVC 구성 전환	21
7.4.8. ViewResolver 재정의	21
7.5. HTTP 클라이언트	21
7.5.1. RestTemplate 가 프록시를 사용하도록 구성	21
7.6. 로깅	21
7.6.1. Logback 구성	21
7.6.2. Log4j 구성	21
7.7. 데이터 접근	22
7.7.1. DataSource 구성	22
7.7.2. DataSource 복수 정의	22
7.7.3. 스프링 Data 리파지토리 이용	22
7.7.4. 스프링 구성으로부터 @Entity 정의 분리	22
7.7.5. JPA 속성 구성	22
7.7.6. EntityManagerFactory 재정의 이용	22
7.7.7. 복수 EntityManager 이용	22
7.7.8. 전통적인 persistence.xml 이용	22
7.7.9. 스프링 Data JPA 와 몽고 리파지토리 이용	22
7.7.10. REST 종단점 로 스프링 Data 리파지토리 노출	22
7.8. 데이터베이스 초기화	22

7.8.1. JPA 를 이용한 데이터베이스 초기화	22
7.8.2. Hibernate 를 이용한 데이터베이스 초기화	22
7.8.3. 스프링 JDBC 를 이용한 데이터베이스 초기화	22
7.8.4. 스프링 배치 를 이용한 데이터베이스 초기화	22
7.8.5. 고차원 데이터베이스 마이그레이션 도구 사용	22
7.9. 배치 애플리케이션	22
7.9.1. 구동시 스프링 배치 작업 실행	22
7.10. 액츄에이터	23
7.10.1. 액츄에이터 종단점 의 포트 혹은 주소 변경	23
7.10.2. whitelabel 오류 페이지 재정의	23
7.10.3. 액츄에이터 과 Jersey	23
7.11. 보안	23
7.11.1. 스프링 부트 보안 구성 전환	23
7.11.2. AuthenticationManager 변경하고 사용자 계정 추가	23
7.11.3. 프록시 서버 뒤에서 실행될 때 HTTPS 활성화	23
7.12. 핫스와핑(Hot swapping)	23
7.12.1. 정적 콘텐츠 재적재	23
7.12.2. 컨테이너 재시작없이 템플릿 재적재	23
7.12.3. 빠른 애플리케이션 재시작	23
7.13. 빌드	23
7.13.1. 빌드 정보 생성	23
7.13.2. 깃 정보 생성	23
7.13.3. 의존성 버전 재정의	23
7.13.4. Java 6 에서 실행하기	23
7.14. 전통적 배포	23
7.14.1. 배포가능한 war 파일 생성	23
7.14.2. 오래된 서블릿 컨테이너에 배포가능한 war 파일 생성	23
7.14.3. 기존 애플리케이션을 스프링 부트 로 변환	23
7.14.4. 오래된 컨테이너(Servlet 2.5)에 war 배포	24
8. 정리	25
부록	25
Appendix A: 개발환경 설정	26
JDK설치	26
STS 설치	26
그레이들 설치	26
Appendix B: 개발 시 참고사항	27
스프링부트 레퍼런스 문서 참고	27
관련 자동구성 클래스 확인	27
debug=true 를 이용한 활성화된 자동설정과 비활성화된 자동설정 확인	27
실행가능한 jar 구조	27
공통 애플리케이션 속성들	27
Appendix C: 참고문헌	28
Spring Framework	28
Spring Boot	28
Spring Security	28
Spring Test	28

스프링 부트를 이해하고 활용하는데 쓸 수 있는 안내서

저자 소개

안녕하세요, 김지현입니다. 항상 자기소개를 할 때면 수줍습니다.

들어가며

스프링 부트가 소개된지... 초반의 불안과 불신을 모두 이겨내고...

책에서 다루는 내용

[시작하기](#)

[스프링 부트 이용](#)

[스프링 부트 기능](#)

[스프링 부트 액추에이터: 출시준비기능](#)

[스프링 부트 애플리케이션 배포](#)

[스프링 부트 애플리케이션 배포](#)

[빌드 도구 플러그인](#)

[스프링 부트 활용 안내](#)

[정리](#)

책을 따라하기 위해 필요항목

JDK8, IDE(STS, IntelliJ, Command)

대상독자

스프링 부트 를 기반으로 한 개발에 관심을 가지고 이제 막 예제를 만들어보려 하거나 스프링 프레임워크에 대한 기본적인 지식이 있는 개발자라면..

관례(Convention)

책을 읽어가다보면 반복적으로 사용되는 표현들이 있을 것이다. 코드는 다음과 같이,

```
public class Honeymon {
    private String firstName;
    private String lastName;
    private Gender gender;
    private Integer age;
    private String site;
    private Integer weight;

    public Fat eat(Food food) {
        this.weight += food.getWeight();
    }
}
```

코드 중에서 중요한 부분에 대해서는 다음과 같이,

```
public class Honeymon {
    private String firstName;
    private String lastName;
    private Gender gender;
    private Integer age;
    private String site;
    private Integer weight;

    public Fat eat(Food food) {
        this.weight += food.getWeight(); ①
    }
}
```

① 먹은 만큼 찐다! 만고불변의 법칙!

중요한 단어나 문장에 대해서는 **굵은 글씨**로 처리한다. **스프링 부트**는 프레임워크다. 문장 중간에 노출되는 클래스명과 같은 코드성 표현에 대해서는 **Honeymon**으로 표현한다.

기억해두었으면 하는 부분들에 대해서는 다음과 같이 **노트**로 표기한다.

NOTE 노트

알아두면 유용한 것에 대해서는 다음과 같이 **유용함**으로 표기한다.

TIP 유용함!

- 인용한 것에 대해서는 **인용**으로 표기한다.

인용

용어나 개념을 소개하는 경우에는 한국어로 표현하는 것을 선호하는 편이다. 첫 표기시에는 **자동구성(Configuration)**과 **설정(SetUp)**과 같이 한국어와 영어를 병행표기하고 이후 표기에서는 한국어만 표기한다. 적절한 한국어 표현을 찾기 어려운 경우에는 **액추에이터(Actuator)** 외래어 표기법을 따른다.

이 표기에 대해서는 의견이 분분하다. 어떤 이들은 영어를 그대로 표기하는 게 좋다하는 사람도 있지만, 나는 억지스러울지도 모르겠지만 한국어로 표기하고자 하는 노력을 기울이고 있다. 그렇다고 해서 나만 이해할 수 있는 엉터리 표기는 피하려고 노력한다. 이에 대해서는 각자의 방식으로 접근하고 가장 효율적인 방법을 선택하는 것이 적합하다고 본다.

예제

이 책에서 사용된 예제는 <https://github.com/ihoneymon/boot-spring-boot> 에서 내려받을 수 있다. 메이븐이나 그레이들 없이 실행할 수 있도록 래퍼(wrapper)가 구성되어 있지만 이 래퍼를 실행하기 위해서는 JVM이 있어야 한다. 코드의 기본구성은 JDK8을 기준으로 작성되어 있다. 필요하다면 리파지토리를 복제(clone)해서 JDK7에서도 실행가능하게 구성하고 공유해주면 다른 개발자들에게 도움이 될 것이다.

스프링 부트

Spring Boot makes it easy to create stand-alone, production-grade Spring based Applications that you can "just run". We take an opinionated view of the Spring platform and third-party libraries so you can get started with minimum fuss. Most Spring Boot applications need very little Spring configuration.

기능들:

- Create stand-alone Spring applications
- Embed Tomcat, Jetty or Undertow directly (no need to deploy WAR files)
- Provide opinionated 'starter' POMs to simplify your Maven configuration Automatically configure Spring whenever possible
- Provide production-ready features such as metrics, health checks and externalized configuration
- Absolutely no code generation and no requirement for XML configuration

Chapter 1. 시작하기

1.1. 스프링 부트 소개

1.2. 시스템 요구사항

1.3. 스프링 부트 설치

1.3.1. 설치방법 소개

스프링 부트 CLI 를 사용한 방법은 생략한다. 내가 많이 안쓴다. ===== 메이븐 설치 ===== 그레이들 설치

1.4. 스프링 부트 애플리케이션 개발

1.4.1. 프로젝트 생성

1.4.2. 의존성 추가

1.4.3. 코드 작성하기

1.4.4. 실행하기

1.4.5. 실행가능한 jar 생성

1.5. 이어질 이야기

Chapter 2. 스프링 부트 이용

2.1. 빌드 시스템

2.1.1. 의존성 관리

2.1.2. 메이븐

2.1.3. 그레이들

2.1.4. 스타터

2.2. 코드 구조

2.2.1. "기본" 패키지 사용

2.2.2. 메인 애플리케이션 클래스 위치

2.3. 구성(Configuration) 클래스

2.3.1. 추가적인 구성클래스 불러오기

2.3.2. XML 구성 불러오기

2.4. 자동구성(Auto-configuration)

2.4.1. 자동구성 대체하기

2.4.2. 특정 자동구성 비활성화하기

2.5. 스프링 빈과 의존성 주입

2.6. @SpringBootApplication 애노테이션 사용하기

2.7. 애플리케이션 실행하기

2.7.1. IDE 에서 실행하기

2.7.2. 패키징된 애플리케이션 실행하기

2.7.3. 메이븐 플러그인으로 실행하기

2.7.4. 그레이들 플러그인으로 실행하기

2.7.5. 핫 스와핑

2.8. 스프링 개발자도구

2.8.1. 기본 속성들

2.8.2. 자동 재시작

2.8.3. 전역 설정

2.8.4. 원격 애플리케이션

2.9. 출시를 위한 애플리케이션 패키징

2.10. 이어질 이야기

Chapter 3. 스프링 부트 기능

3.1. 스프링애플리케이션(SpringApplication)

3.1.1. 배너 재정의

3.1.2. SpringApplication 재정의

3.1.3. 유연한(Fuent) 빌더 API

3.1.4. 애플리케이션 이벤트 및 리스너

3.1.5. 웹 환경

3.1.6. 애플리케이션 실행 인자에 접근

3.1.7. ApplicationRunner 혹은 CommandLineRunner 사용하기

3.1.8. 애플리케이션 종료

3.1.9. 관리자 기능

3.2. 외부 구성

3.2.1. 랜덤값 구성

3.2.2. 명령줄 속성 접근

3.2.3. 애플리케이션 속성 파일

3.2.4. 프로파일 정의 속성

3.2.5. 속성 내 치환자(placeholder)

3.2.6. properties 를 대신하여 YAML 사용하기

YAML 적재

{spring-environment} 에서 properties 처럼 `YAML` 사용하기

YAML 문서로 다중 프로파일 다루기

YAML 단점

YAML 목록 병합(Merging)

3.2.7. 보장형 구성 속성

서드파티 구성

느슨한 연결

Properties 관례

@ConfigurationProperties 유효성검사

@ConfigurationProperties 대 @Value

3.3. 프로파일

3.3.1. 활성화할 프로파일 추가하기

3.3.2. 프로파일을 프로그래밍적으로 설정

3.3.3. 프로파일 정의 구성 파일

3.4. 로깅

3.4.1. 로그 형식

3.4.2. 콘솔 출력

3.4.3. 파일 출력

3.4.4. 로그 레벨

3.4.5. 로그 구성 재정의

3.4.6. 로그백 확장

3.5. 웹애플리케이션 개발

3.5.1. 'Spring Web MVC framework'

3.5.2. 내장 서블릿 컨테이너 지원

서블릿, 필터 와 리스너

서블릿 컨텍스트 초기화

EmbeddedWebApplicationContext

내장 서블릿 컨테이너 재정의

JSP 제한

3.6. 보안

3.6.1. OAuth2

공인(Authorization) 서버

자원(Resource) 서버

3.6.2. 사용자 정보 속 토큰 유형

3.6.3. 사용자정보 RestTemplate 재정의

클라이언트

Single Sign On

3.6.4. 액츄에이터 보안(Actuator Security)

3.7. SQL 데이터베이스 이용

3.7.1. DataSource 구성

3.7.2. JdbcTemplate 사용

3.7.3. JPA 그리고 '스프링 Data'

3.7.4. H2 사용

프로파일 구성

H2 웹콘솔 사용

3.7.5. jOOQ 사용

3.8. NoSQL 기술 이용

3.8.1. 레디스(Redis)

3.8.2. 몽고DB(MongoDB)

몽고DB 데이터베이스 연결

MongoTemplate

스프링 Data 몽고DB 리파지토리

내장 몽고

3.8.3. 솔(Solr)

솔 연결하기

스프링 Data 솔 리파지토리

3.8.4. 엘라스틱서치(Elasticsearch)

스프링 Data 를 통해 엘라스틱서치 연결

스프링 Data 엘라스틱서치

3.9. 캐시

3.9.1. 지원하는 캐시 제공자(Provider)

Generic

JCache

EhCache 2.x

Hazelcast

Redis

Guava

Simple

None

3.10. REST 서비스 호출

3.10.1. RestTemplate 재정의

3.11. 스프링 세션

3.12. 테스트

3.12.1. 테스트 관련 의존성

3.12.2. 스프링 애플리케이션 테스트

3.12.3. 스프링 부트 애플리케이션 테스트

테스트 구성 탐색

테스트 구성 제외

랜덤포트 이용

빈에 대한 목(Mock) 과 스파이(Spy) 처리

자동구성된 테스트

자동구성된 JSON 테스트

자동구성된 스프링 MVC 테스트

자동구성된 Data JPA 테스트

자동구성된 REST 클라이언트

자동구성된 스프링 REST Docs 테스트

스폭(Spock) 을 사용하여 스프링 부트 애플리케이션 테스트

3.12.4. 테스트 유틸리티

ConfigFileApplicationContextInitializer

EnvironmentTestUtils

OutputCapture

TestRestTemplate

3.13. 웹소켓(WebSocket)

3.14. 웹 서비스(Web Service)

3.15. 자동구성 직접 생성하기

3.15.1. 자동구성 빈 이해하기

3.15.2. 자동구성 대상자 위치

3.15.3. 조건 애너테이션

클래스 조건

빈 조건

속성 조건

자원 조건

웹애플리케이션 조건

SpEL 표현식 조건

3.15.4. 스타터 직접 생성하기

이름짓기

자동구성 모듈

스타터 모듈

3.16. 이어질 이야기

Chapter 4. 스프링 부트 액츄에이터: 출시준비기능

4.1. 출시준비 기능 활성화

4.1.1. 종단점(Endpoint)

4.1.2. 종단점 재정의

4.1.3. 액츄에이터 MVC 종단점 를 위한 하이퍼미디어

4.1.4. CORS 지원

4.1.5. 재정의 종단점 추가하기

4.1.6. 건강정보

4.1.7. `HealthIndicator` 를 사용한 보안

4.1.8. 애플리케이션 정보

자동구성된 `InfoContributors`

재정의한 애플리케이션 정보

깃(Git) 커밋 정보

빌드(build) 정보

`InfoContributors` 재정의 작성

4.2. HTTP 로 모니터링하고 관리하기

4.2.1. 종단점 에 대한 노출강도 보안관리

4.2.2. 관리 종단점 경로 재정의

4.2.3. 관리 서버 포트 재정의

4.2.4. 관리를 위한 SSL 재정의

4.2.5. 관리 서버 주소 재정의

4.2.6. HTTP 종단점 비활성화

4.2.7. HTTP 건강 종단점 접근 제한

4.3. 원격셀을 통해 모니터링하고 관리하기

4.3.1. 원격셀로 접속

4.3.2. 원격셀 확장

4.3.3. 원격셀 플러그인

4.4. 측량(Metrics)

4.4.1. 시스템 측량

4.4.2. 데이터소스 측량

4.4.3. 캐시 측량

4.4.4. 톰캣 세션 측량

4.4.5. 측량 기록

4.4.6. 공개 측량 추가

4.5. 감시

4.6. 추적

4.7. 프로세스 모니터링

4.7.1. 확장 구성

4.7.2. 프로그래밍적인 처리

4.8. 이어질 이야기

Chapter 5. 스프링 부트 애플리케이션 배포

5.1. 클라우드 배포

5.1.1. 클라우드 파운드리(Cloud Foundry)

5.1.2. 히로쿠(Heroku)

5.1.3. 오픈시프트(OpenShift)

5.1.4. 아마존웹서비스(AWS, Amazon Web Service)

5.1.5. 구글앱엔진

5.2. 스프링 부트 애플리케이션 설치

5.2.1. 유닉스/리눅스 서비스

5.3. 마이크로소프트 윈도우즈 서비스

5.4. 이어질 이야기

Chapter 6. 빌드 도구 플러그인

6.1. 스프링 부트 메이븐 플러그인

6.1.1. 플러그인 포함

6.1.2. 실행가능한 jar 와 war 파일 패키징

6.2. 스프링 부트 그레이들 플러그인

6.2.1. 플러그인 포함

6.2.2. 그레이들 의존성 관리

6.2.3. 실행가능한 jar 와 war 파일 생성

6.2.4. 스프링 부트 플러그인 구성

6.2.5. 리패키징 구성

6.2.6. 그레이들 플러그인이 어떻게 동작하는지 이해하기

6.2.7. 그레이들 을 이용해서 메이븐 리파지토리에 아티팩트 배포

6.3. 다른 빌드 시스템 지원

6.3.1. 리패키징 아카이브

6.3.2. 내포된 라이브러리

6.3.3. 메인 클래스 찾기

6.3.4. 리패키지 구현 예제

6.4. 이어질 이야기

Chapter 7. 스프링 부트 활용 안내

7.1. 스프링 부트 애플리케이션

7.1.1. 자동구성 문제해결

7.1.2. `Environment` 혹은 `ApplicationContext` 가 시작하기 전에 재정의하기

7.1.3. `ApplicationContext` 계층 조직(부모 혹은 루트 컨텍스트 추가)

7.1.4. 비-웹애플리케이션 생성

7.2. 속성 및 구성

7.2.1. 빌드 시 자동으로 속성 확장

7.2.2. `SpringApplication` 외부 구성

7.2.3. 애플리케이션 속성 외장파일 위치변경

7.2.4. 명령줄 인자 축약사용

7.2.5. 외부속성에 대한 YAML 사용

7.2.6. 스프링 프로파일 활성화

7.2.7. 환경 의존적인 구성 변경

7.2.8. 외부속성에 대한 빌트인 옵션(built-in option) 발견

7.3. 내장서블릿 컨테이너

7.3.1. 애플리케이션에 서블릿, 필터 혹은 리스너 추가하기

7.3.2. HTTP 포트 변경

7.3.3. 할당되지 않은 HTTP 포트 무작위 사용

7.3.4. 실행시 HTTP 포트 발견

7.3.5. SSL 구성

7.3.6. 접근로그 구성

7.3.7. 프론트엔드 프록시 서버 이용

7.3.8. 톰캣 구성

7.3.9. 톰캣을 통한 다중 커넥터 활성화

7.3.10. 톰캣 7.x 혹은 8.0 이용

7.3.11. `@ServerEndpoint` 를 사용한 웹소켓 종단점 생성

7.3.12. HTTP 응답 압축 활성화

7.4. 스프링 MVC

7.4.1. JSON REST 서비스 작성

7.4.2. XML REST 서비스 작성

7.4.3. 잭슨(Jackson) 오브젝트매퍼(`ObjectMapper`) 재정의

7.4.4. `@ResponseBody` 랜더링 재정의

7.4.5. Multipart 파일 업로드 제어

7.4.6. 스프링 MVC `DispatcherServlet` 전환

7.4.7. 기본 MVC 구성 전환

7.4.8. `ViewResolver` 재정의

7.5. HTTP 클라이언트

7.5.1. `RestTemplate` 가 프록시를 사용하도록 구성

7.6. 로깅

7.6.1. Logback 구성

7.6.2. Log4j 구성

7.7. 데이터 접근

7.7.1. DataSource 구성

7.7.2. DataSource 복수 정의

7.7.3. 스프링 Data 리파지토리 이용

7.7.4. 스프링 구성으로부터 @Entity 정의 분리

7.7.5. JPA 속성 구성

7.7.6. EntityManagerFactory 재정의 이용

7.7.7. 복수 EntityManager 이용

7.7.8. 전통적인 persistence.xml 이용

7.7.9. 스프링 Data JPA 와 몽고 리파지토리 이용

7.7.10. REST 종단점 로 스프링 Data 리파지토리 노출

7.8. 데이터베이스 초기화

7.8.1. JPA 를 이용한 데이터베이스 초기화

7.8.2. Hibernate 를 이용한 데이터베이스 초기화

7.8.3. 스프링 JDBC 를 이용한 데이터베이스 초기화

7.8.4. 스프링 배치 를 이용한 데이터베이스 초기화

7.8.5. 고차원 데이터베이스 마이그레이션 도구 사용

Flyway

Liquibase

7.9. 배치 애플리케이션

7.9.1. 구동시 스프링 배치 작업 실행

7.10. 액츄에이터

7.10.1. 액츄에이터 종단점 의 포트 혹은 주소 변경

7.10.2. `whitelabel` 오류 페이지 재정의

7.10.3. 액츄에이터 과 Jersey

7.11. 보안

7.11.1. 스프링 부트 보안 구성 전환

7.11.2. `AuthenticationManager` 변경하고 사용자 계정 추가

7.11.3. 프록시 서버 뒤에서 실행될 때 HTTPS 활성화

7.12. 핫스와핑(Hot swapping)

7.12.1. 정적 콘텐츠 재적재

7.12.2. 컨테이너 재시작없이 템플릿 재적재

7.12.3. 빠른 애플리케이션 재시작

7.13. 빌드

7.13.1. 빌드 정보 생성

7.13.2. 깃 정보 생성

7.13.3. 의존성 버전 재정의

7.13.4. Java 6 에서 실행하기

7.14. 전통적 배포

7.14.1. 배포가능한 war 파일 생성

7.14.2. 오래된 서블릿 컨테이너에 배포가능한 war 파일 생성

7.14.3. 기존 애플리케이션을 스프링 부트 로 변환

7.14.4. 오래된 컨테이너(Servlet 2.5)에 war 배포

Chapter 8. 정리

부록

Appendix A: 개발환경 설정

JDK설치

STS 설치

그레이들 설치

Appendix B: 개발 시 참고사항

스프링부트 레퍼런스 문서 참고

관련 자동구성 클래스 확인

`debug=true` 를 이용한 활성화된 자동설정과 비활성화된 자동설정
확인

실행가능한 jar 구조

공통 애플리케이션 속성들

Appendix C: 참고문헌

Spring Framework

Spring Boot

Spring Security

Spring Test